Heaps & Priority Queues

Dr. Nafis Sadeq
Assistant Professor
Department of Computer Science and Engineering
East West University



Outline

1. Priority Queues
2. Binary Heaps
3. Max-Heapify
4. Build-Heap

Priority Queue: Idea

A **priority queue** is a queue ordered by a **priority value** (key).

- Standard queue: enqueue at rear, dequeue from front.
- Priority queue: dequeue removes the **highest priority** element.

Why useful?

When "who goes next" depends on importance/urgency, not arrival time.

Applications of Priority Queues

Examples:

- Boarding line of planes at an airport
- Operating systems scheduling (priority-based)
- Hospital emergency rooms.

Max-Priority Queue Operations

Max-Priority Queue (Max-PQ): dequeue item with highest key.

- `Insert(S, x)`: insert element x into S
- `Maximum(S)`: return (do not remove) largest key
- `Extract-Max(S)`: remove and return largest key
- `Increase-Key(S, x, k)`: increase x's key to k

Goal

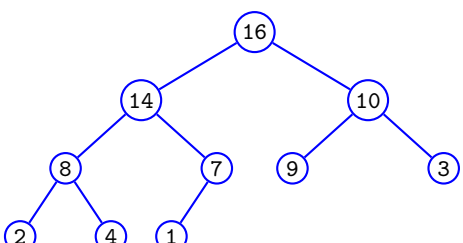
Support these efficiently (typically $\mathcal{O}(\log n)$).

Binary Heaps

- The (binary) heap data structure is an array object that can be viewed as a complete binary tree
  - Each node of the tree corresponds to an element of the array that stores the value in the node.
  - The tree is completely filled on all levels except possibly the lowest, where it is filled from the left up to a point.

$A =$

16	14	10	8	7	9	3	2	4	1



Heap as an Array

To represent a complete binary tree as an array (1-indexed):

- Root is $A[1]$
- Node i is $A[i]$
- Parent$(i) = \lfloor i/2 \rfloor$
- Left$(i) = 2i$
- Right$(i) = 2i + 1$

Why 1-indexed in pseudocode?

The formulas become very clean.

Min-Heap vs Max-Heap

Types of binary heaps:

- **Min-Heap**
  - Root $\leq$ all nodes in subtrees
  - Each subtree is also a min-heap
- **Max-Heap**
  - Root $\geq$ all nodes in subtrees
  - Each subtree is also a max-heap

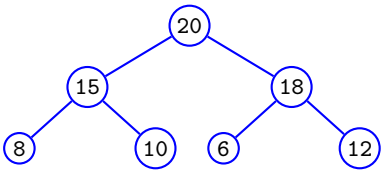
Priority Queue Connection

Max-heap supports `Maximum` and `Extract-Max` efficiently.

Heap Property

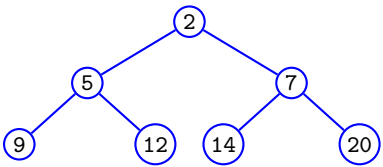
Max-Heap

- Largest element is stored at the root
- $A[\text{Parent}(i)] \geq A[i]$ for all $i > 1$



Min-Heap

- Smallest element is stored at the root
- $A[\text{Parent}(i)] \leq A[i]$ for all $i > 1$



Max-Heapify()

Max-Heapify maintains the max-heap property.

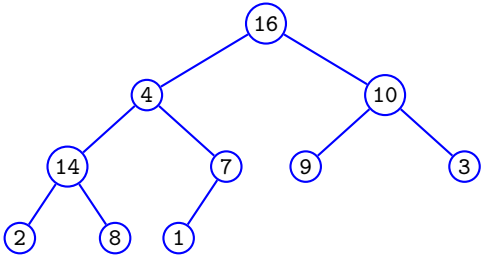
- Given a node i with children l and r
- Assume subtrees rooted at l and r are already max-heaps
- The subtree rooted at i may violate the heap property
- Action: let the value at i "float down" until restored

Max-Heapify(A, i)

Max-Heapify(A, i)

```
l ← 2i
r ← 2i + 1
largest ← i
if l ≤ heap-size[A] and A[l] > A[i]
      largest ← l
if r ≤ heap-size[A] and A[r] > A[largest]
      largest ← r
if largest ≠ i
      exchange A[i] with A[largest]
      Max-Heapify(A, largest)
```

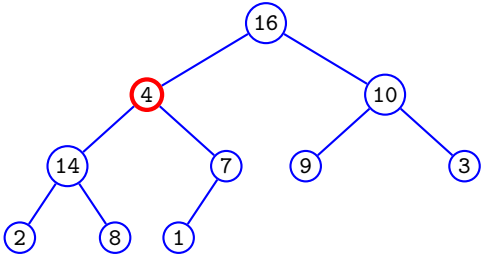
Heapify() Example



$A =$

16	4	10	14	7	9	3	2	8	1

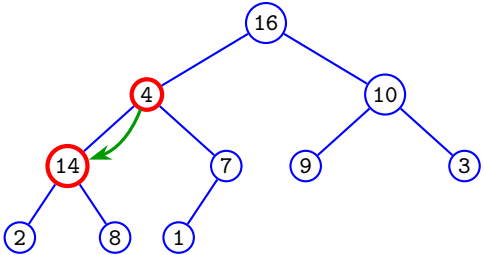
Heapify() Example



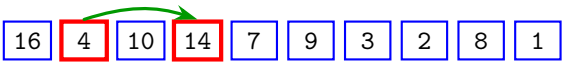
$A =$

16	4	10	14	7	9	3	2	8	1

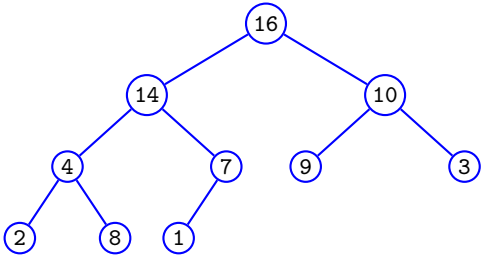
Heapify() Example



$A =$



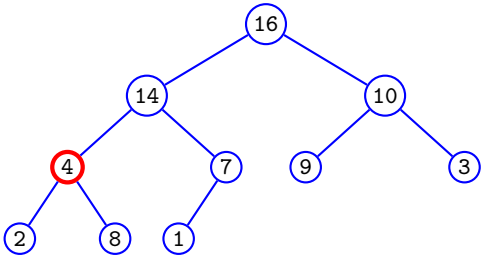
Heapify() Example



$A =$

16	14	10	4	7	9	3	2	8	1

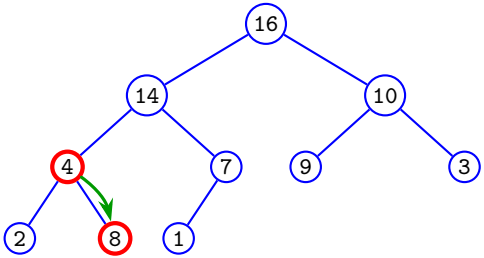
Heapify() Example



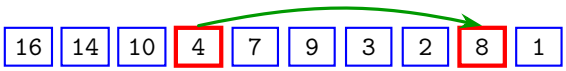
$A =$

16	14	10	4	7	9	3	2	8	1

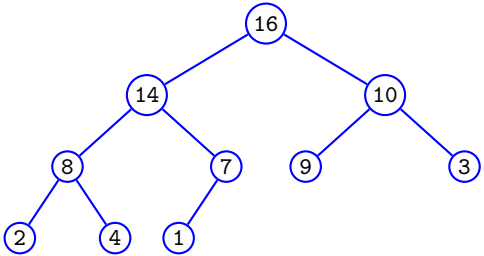
Heapify() Example



$A =$



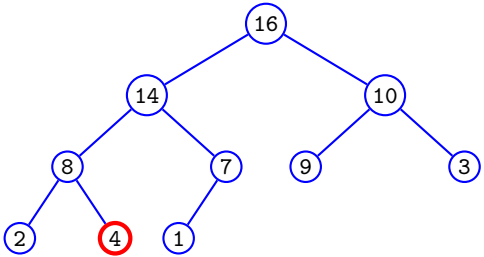
Heapify() Example



$A =$

16	14	10	8	7	9	3	2	4	1

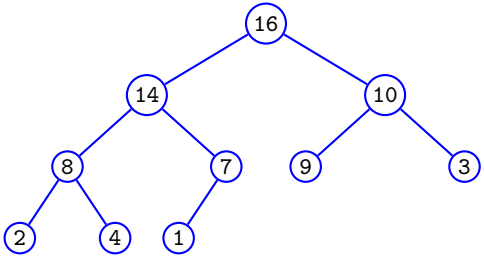
Heapify() Example



$$A =$$

16	14	10	8	7	9	3	2	4	1

Heapify() Example



$A =$

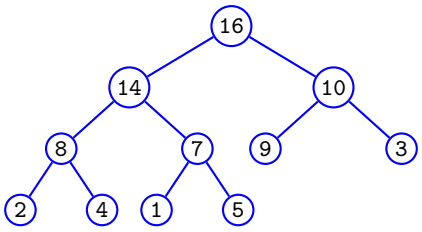
16	14	10	8	7	9	3	2	4	1

Analyzing Heapify()

- Aside from the recursive call, what is the running time of `Heapify()`?
- How many times can `Heapify()` recursively call itself?
- What is the worst-case running time of `Heapify()` on a heap of size n?

Analyzing Heapify(): Formal

- Fixing up relationships among the elements $A[i]$, $A[l]$, and $A[r]$ takes $\Theta(1)$ time
- *If the heap at i has n elements, at most how many elements can the subtrees at l or r have?*



- Answer: $2n/3$ (worst case: bottom row half full)
- So time taken by `Heapify()` is given by

$$T(n) \leq T(2n/3) + \Theta(1)$$

Analyzing Heapify(): Solving the Recurrence

- From the analysis we have the recurrence

$$T(n) \leq T(2n/3) + \Theta(1)$$

- Expand the recurrence:

$$T(n) \leq T\left((2/3)n\right) + c$$

$$T(n) \leq T\left((2/3)^2 n\right) + 2c$$

$$T(n) \leq T\left((2/3)^k n\right) + kc$$

- The recursion stops when

$$(2/3)^k n = 1$$

Analyzing Heapify(): Solving the Recurrence

- Solving for k:

$$(2/3)^k n = 1$$

$$k = \log_{3/2} n = \Theta(\log n)$$

- Therefore

$$T(n) = O(\log n)$$

Analyzing Heapify(): Simple Intuition

- Each recursive call moves the element down one level in the heap.
- A binary heap of size n has height

$$h = \lfloor \log_2 n \rfloor$$

- Therefore, in the worst case the element moves down at most h levels.
- Thus the running time of `Heapify()` is

$$O(h) = O(\log n)$$

Heap Operations: BuildHeap()

- We can build a heap in a bottom-up manner by running `Heapify()` on successive subarrays
  - Fact: for array of length n, all elements in the range

    $$A[\lfloor n/2 \rfloor + 1 \ldots n]$$

    are heaps (*Why?*)
- So:
  - Walk backwards through the array from $n/2$ to 1, calling `Heapify()` on each node.
  - Order of processing guarantees that the children of node i are heaps when i is processed.

Heap Operations: BuildHeap()

Converts an unorganized array A into a max-heap.

Build-Max-Heap(A)

```
heap-size[A] ← length[A]
for i ← ⌊length[A]/2⌋ downto 1
       do Max-Heapify(A, i)
```

Why Start from $\lfloor n/2 \rfloor$?

- In an array representation of a heap:

$$\text{Left}(i) = 2i$$

$$\text{Right}(i) = 2i + 1$$

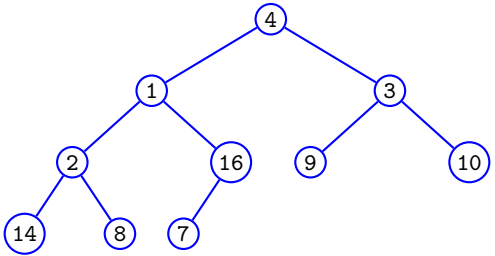
- If $i > n/2$, then $2i > n$
- Therefore node i has no children.
- So all nodes in the range $\lfloor n/2 \rfloor + 1 \ldots n$ are **leaves**.
- Leaves already satisfy the heap property.

BuildHeap() Example

- Work through example

$$A = \{4, 1, 3, 2, 16, 9, 10, 14, 8, 7\}$$

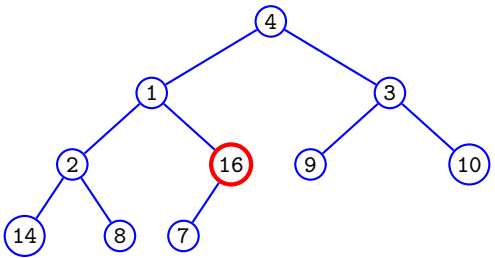
4	1	3	2	16	9	10	14	8	7



BuildHeap() Example (Heapify at $i = 5$)

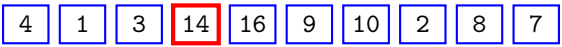
- Start from $i = \lfloor n/2 \rfloor = 5$ (node value 16)

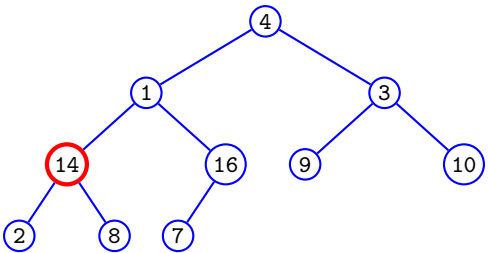
4	1	3	2	16	9	10	14	8	7



BuildHeap() Example (After Heapify at $i = 4$)

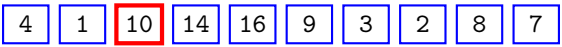
- Heapify at $i = 4$ (node value 2): swap with larger child 14

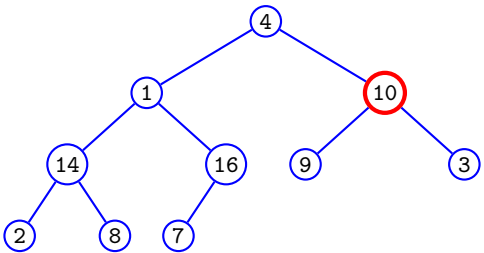




BuildHeap() Example (After Heapify at $i = 3$)

- Heapify at $i = 3$ (node value 3): swap with larger child 10

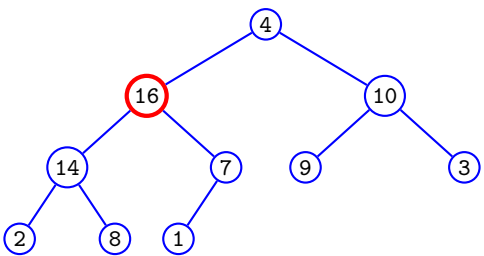




BuildHeap() Example (After Heapify at $i = 2$)

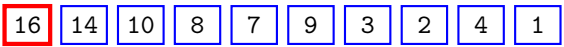
- Heapify at $i = 2$: 1 swaps with 16, then 1 swaps with 7

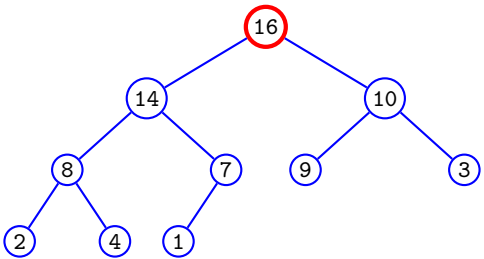
4	16	10	14	7	9	3	2	8	1



BuildHeap() Example (After Heapify at $i = 1$)

- Heapify at $i = 1$ (root): percolate 4 down to restore max-heap





Analyzing BuildHeap()

- Each call to `Heapify()` takes $O(\log n)$ time
- There are $O(n)$ such calls (specifically, $\lfloor n/2 \rfloor$)
- Thus the running time is $O(n \log n)$
  - *Is this a correct asymptotic upper bound?*
  - *Is this an asymptotically tight bound?*
- A tighter bound is $O(n)$
  - *How can this be? Is there a flaw in the above reasoning?*

Analyzing BuildHeap(): Tight

- To `Heapify()` a subtree takes $O(h)$ time, where h is the height of the subtree
  - $h = O(\log m)$, m = number of nodes in the subtree
  - The height of most subtrees is small
- Fact: an n-element heap has at most

$$\left\lceil \frac{n}{2^{h+1}} \right\rceil$$

  nodes of height h
- Prove that `BuildHeap()` takes $O(n)$ time